

Progress report on

ShipmentSure: Predicting On-Time Delivery Using Supplier Data

Name	Batch
Arpita Mishra	03

ShipmentSure: Predicting On-Time Delivery Using Supplier Data

1. Introduction

Our project, “ShipmentSure: Predicting On-Time Delivery Using Supplier Data”, aims to build a machine learning model that can predict whether a shipment will reach on time or not based on supplier and product-related factors. As the first step, we performed Exploratory Data Analysis (EDA) to understand the dataset, explore feature distributions, identify relationships between variables, and check for class imbalance in the target variable (Reached.on.Time_Y.N).

Dataset size: 10,999 records with 12 features (8 numerical, 4 categorical).

Target variable: Reached.on.Time_Y.N (0 = On time, 1 = Late).

2. Univariate Analysis

We examined individual features to understand their distributions:

i. Target Variable:

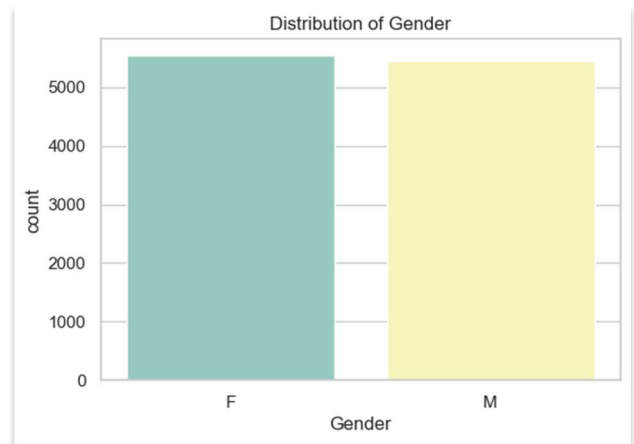
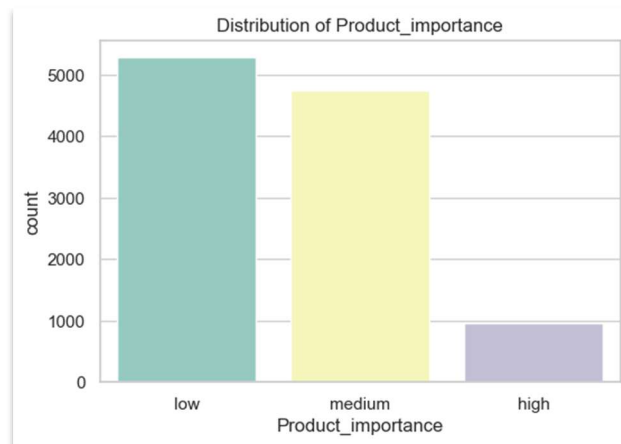
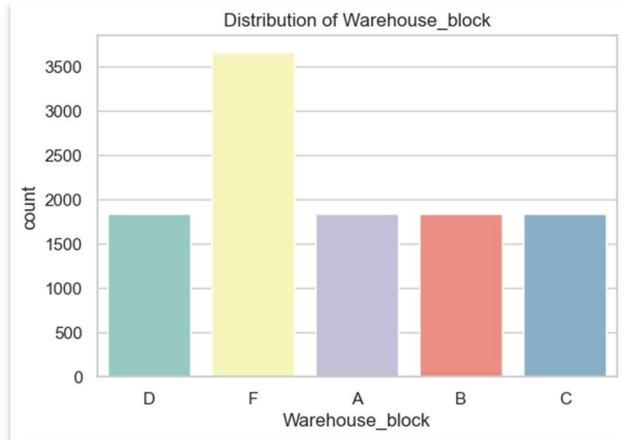
- Class 1 (Late): 6563 (59.7%)
- Class 0 (On Time): 4436 (40.3%)
- Shows mild imbalance toward delayed deliveries.



ii. Categorical Features:

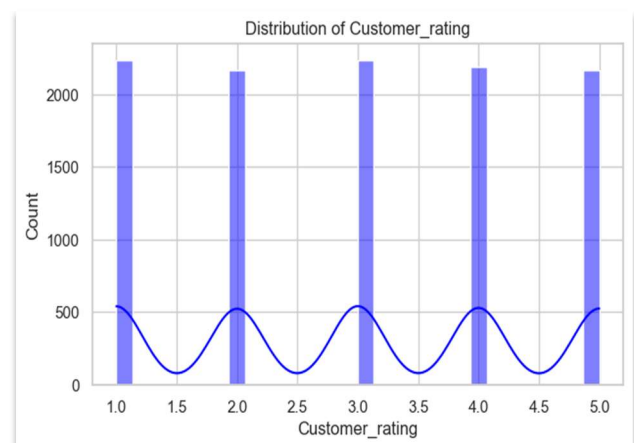
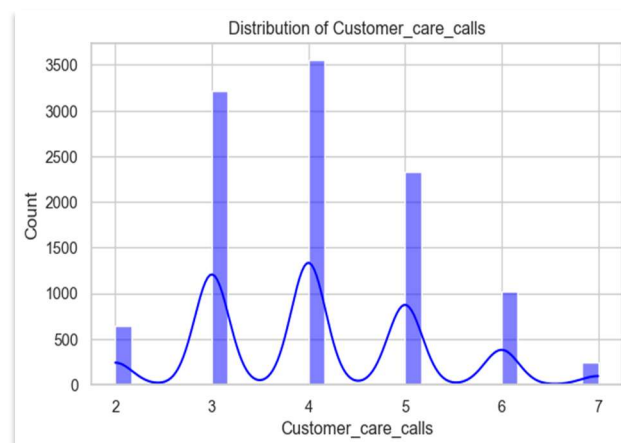
- Warehouse_block: Balanced distribution across blocks A-F.
- Mode_of_Shipment: Mostly “Ship”, followed by “Flight” and “Road”.
- Product_importance: Majority are “low” importance, fewer “medium” and very few “high”.

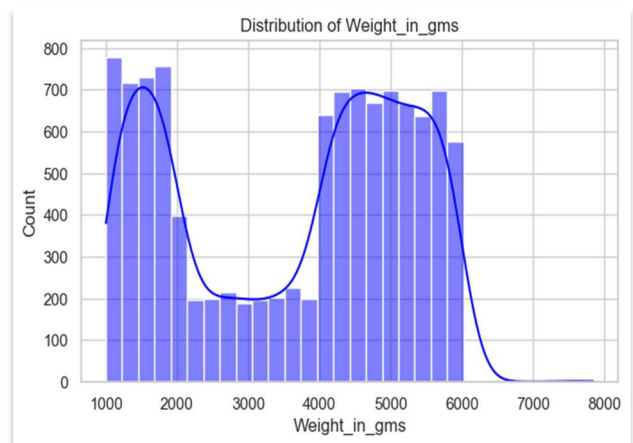
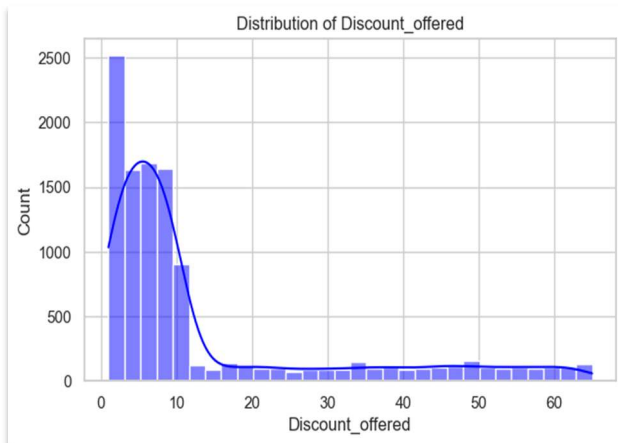
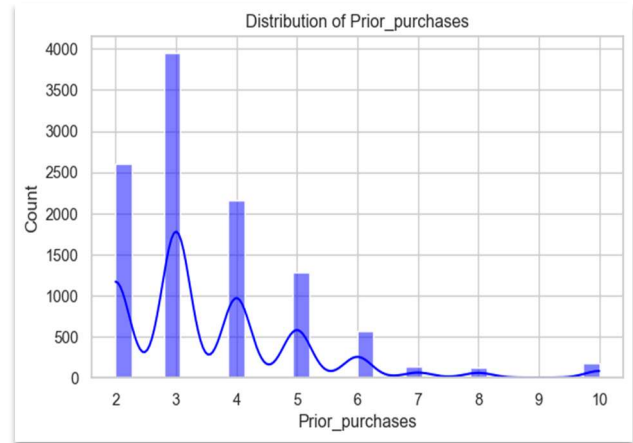
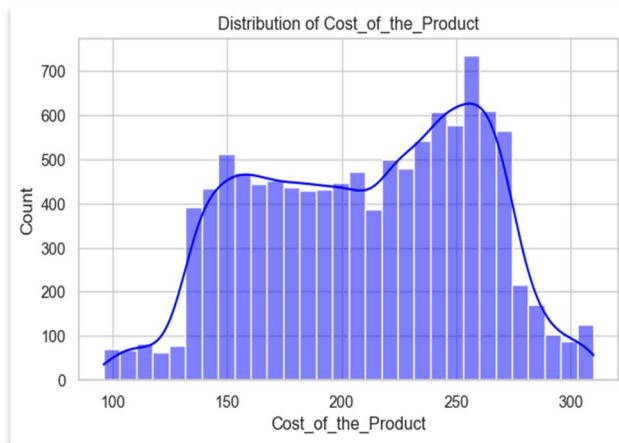
- Gender: Roughly balanced between Male and Female.



iii. Numerical Features:

- Customer_care_calls: Most values between 2–5 calls.
- Customer_rating: Fairly uniform from 1–5.
- Cost_of_the_Product: Wide spread with most values clustered in lower ranges.
- Prior_purchases: Majority have 2–5 past purchases.
- Discount_offered: Skewed toward lower discounts.
- Weight_in_gms: Normally distributed with a long tail.



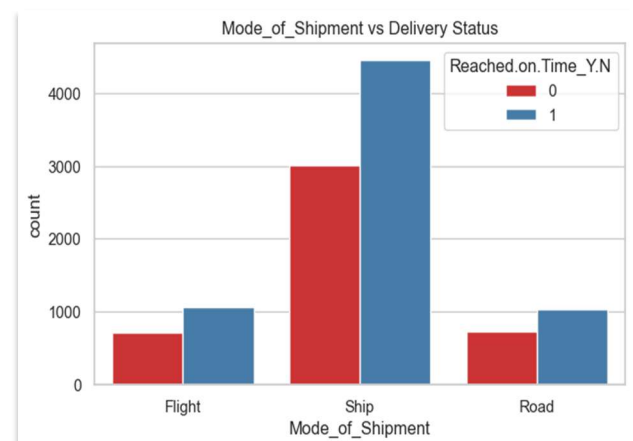
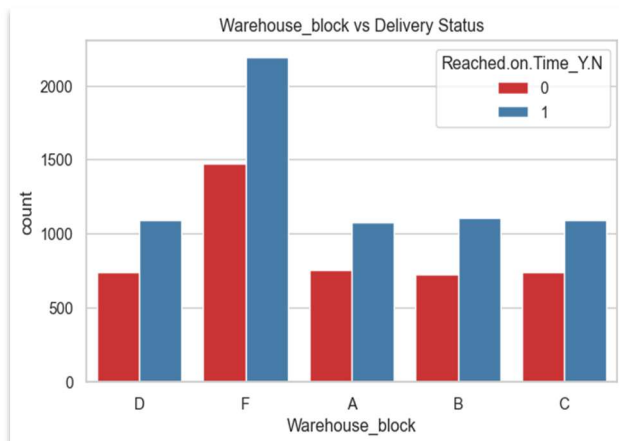


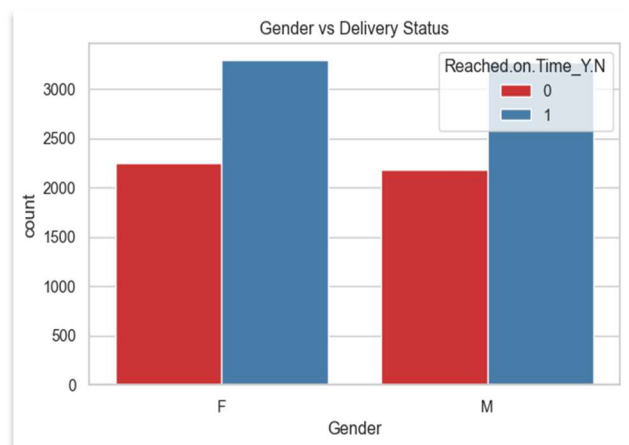
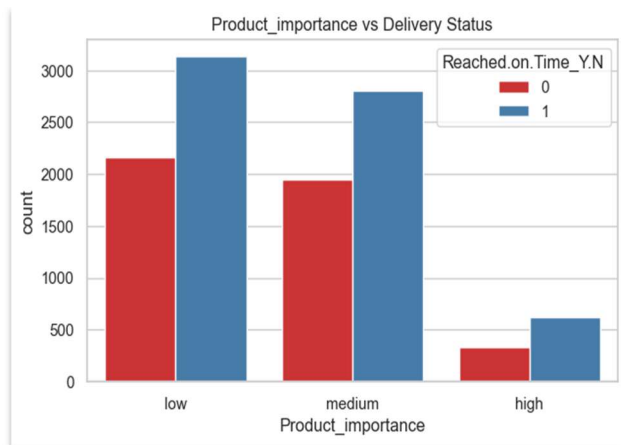
3. Bivariate Analysis

We explored how independent variables relate to the target variable:

i. Categorical vs Target:

- **Mode_of_Shipment:** Flights tend to have better on-time rates compared to shipping by road.
- **Product_importance:** High importance products tend to arrive on time more often.
- **Warehouse_block:** Slight variations, but not strongly predictive on their own.

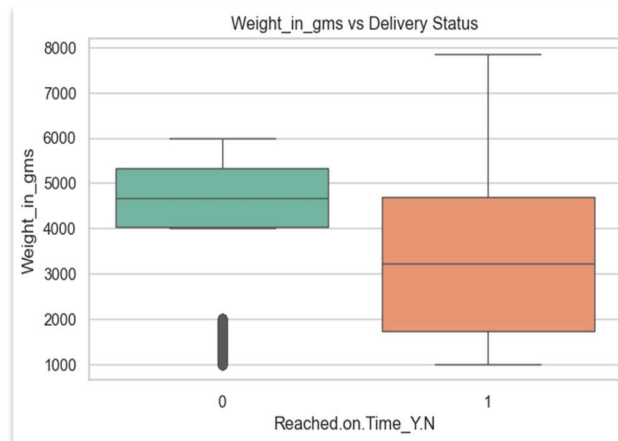




ii. Numerical vs Target:

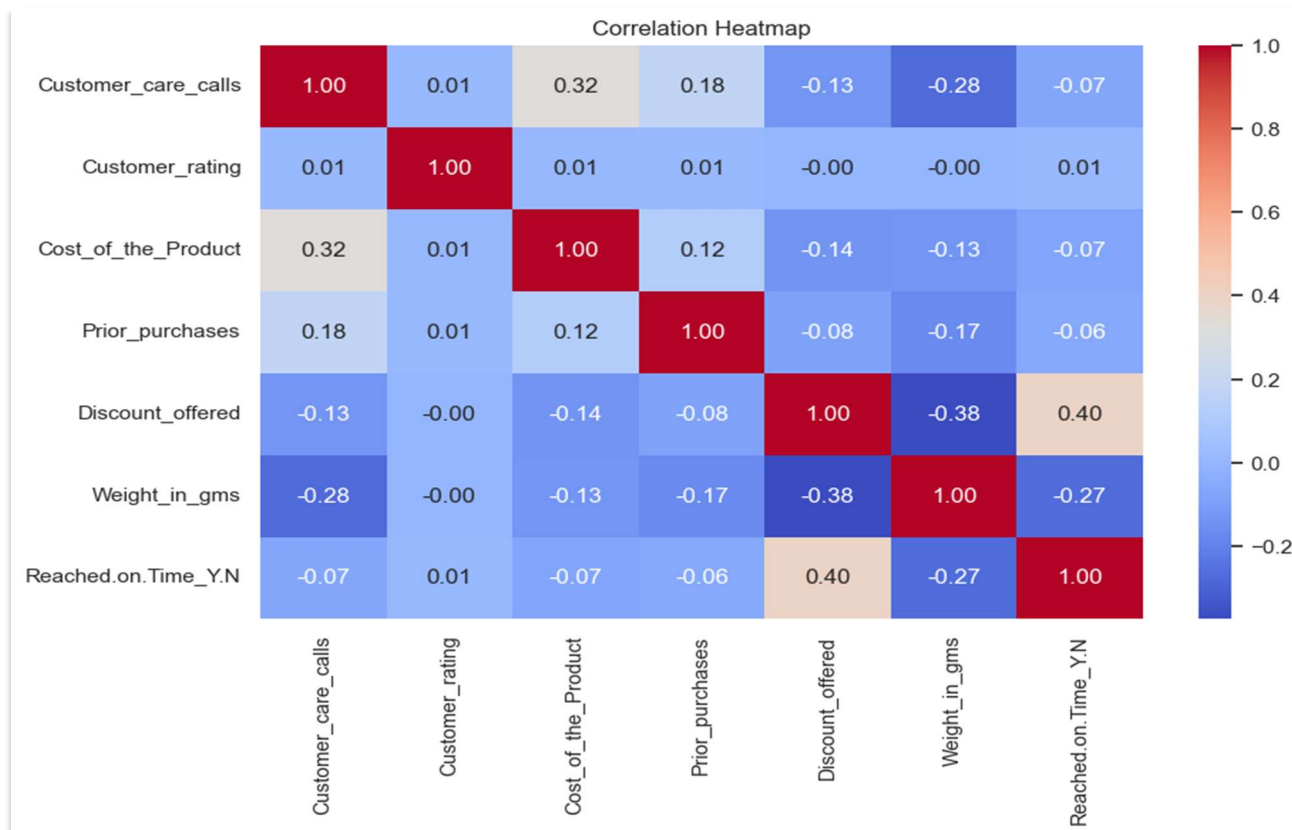
- Discount_offered: Higher discounts are associated with late deliveries.
- Customer_care_calls: Higher calls are linked to late deliveries.
- Weight_in_gms: Heavier products tend to have more delays.
- Customer_rating: Shows little variation with delivery status.





iii. Correlation Heatmap:

- Strong correlation between Weight_in_gms and Cost_of_the_Product.
- Negative relationship between Discount_offered and delivery status (late deliveries linked with higher discounts).



4. Class Imbalance Analysis

We analyzed whether the dataset is balanced for classification:

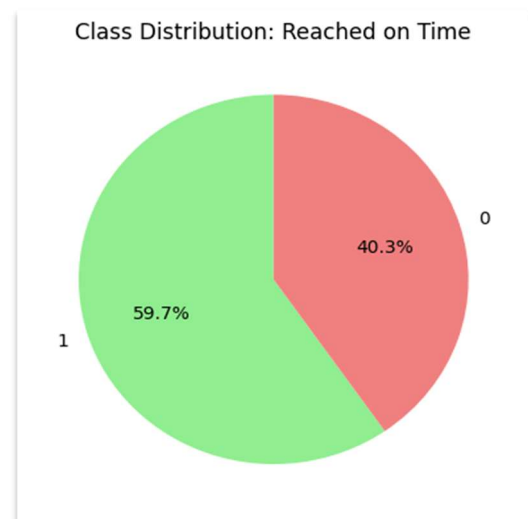
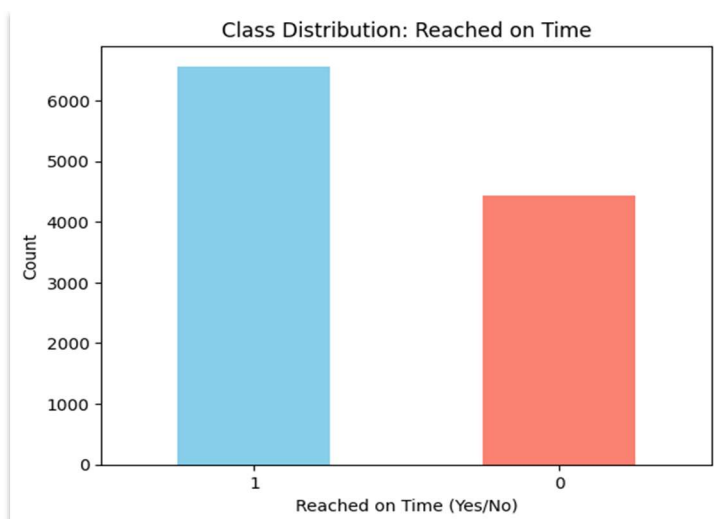
To examine class imbalance in the Reached.on.Time_Yes/No column, means we are checking whether the dataset has a roughly equal number of "Yes" and "No" values—or if one class dominates.

Distribution:

- Class 1 (Late): ~60%
- Class 0 (On Time): ~40%

Visualization:

- Bar plot and pie chart confirm imbalance.



- **Conclusion:**
Not heavily skewed, but imbalance-handling strategies (e.g., SMOTE, class weights) may be useful for modeling.

Data Preprocessing and Dataset Splitting

1. Handling Missing Values

Before applying any machine learning algorithms, it's essential to ensure the dataset is complete and free from inconsistencies. Missing values can lead to biased models or runtime errors. In this project, we performed a thorough check using `df.isnull().sum()` and confirmed that **no columns contained null values**. As a result, no imputation or deletion was necessary, and the dataset was deemed clean and ready for further processing.

2. Encoding Categorical Features

Machine learning models require numerical input, so categorical variables must be transformed appropriately:

- **Label Encoding** was applied to binary or ordinal features such as Gender and Product_importance. This technique assigns a unique integer to each category.
- **One-Hot Encoding** was used for nominal features like Warehouse_block and Mode_of_Shipment. This creates separate binary columns for each category, ensuring no artificial ordering is introduced.

These encoding techniques allow the model to interpret categorical data without misrepresenting relationships between categories.

...	ID	Customer_care_calls	Customer_rating	Cost_of_the_Product	\
0	1	4	2	177	
1	2	4	5	216	
2	3	2	2	183	
3	4	3	3	176	
4	5	2	2	184	
	Prior_purchases	Discount_offered	Weight_in_gms	Reached.on.Time_Y.N	\
0	3	44	1233	1	
1	2	59	3088	1	
2	4	48	3374	1	
3	4	10	1177	1	
4	3	46	2484	1	
	Warehouse_block_B	Warehouse_block_C	Warehouse_block_D	Warehouse_block_F	\
0	False	False	True	False	
1	False	False	False	True	
2	False	False	False	False	
3	True	False	False	False	
4	False	True	False	False	
	Mode_of_Shipment_Road	Mode_of_Shipment_Ship	Product_importance_low	\	
0	False	False	True		
1	False	False	True		
2	False	False	True		
...					
1	False	True			
2	False	True			
3	True	True			
4	True	False			

3. Normalizing Numerical Features

Numerical features often vary widely in scale. For example, **Cost_of_the_Product** may range from **100 to 300**, while **Customer_rating** ranges from **1 to 5**. To prevent features with larger values from dominating the learning process, we applied **StandardScaler**, which transforms each feature to have a mean of 0 and a standard deviation of 1. This ensures that all features contribute equally to the model's learning process.

	ID	Customer_care_calls	Customer_rating	Cost_of_the_Product	Prior_purchases	Discount_offered	Weight_in_gms	Reached.on.Time_Y.N	Warehouse_block_B
0	1	-0.047711	-0.700755	-0.690722	-0.372735	1.889983	-1.468240	1	False
1	2	-0.047711	1.421578	0.120746	-1.029424	2.815636	-0.333893	1	False
2	3	-1.799887	-0.700755	-0.565881	0.283954	2.136824	-0.159002	1	False
3	4	-0.923799	0.006689	-0.711529	0.283954	-0.208162	-1.502484	1	True
4	5	-1.799887	-0.700755	-0.545074	-0.372735	2.013404	-0.703244	1	False

Warehouse_block_B	Warehouse_block_C	Warehouse_block_D	Warehouse_block_F	Mode_of_Shipment_Road	Mode_of_Shipment_Ship	Product_importance_low	Product_importance_medium	Gender_M
False	False	True	False	False	False	True	False	False
False	False	False	True	False	False	True	False	True
False	False	False	False	False	False	True	False	True
True	False	False	False	False	False	False	True	True
False	True	False	False	False	False	False	True	False

4. Splitting the Dataset

To evaluate model performance effectively, the dataset was split into training and testing subsets using `train_test_split` from `sklearn.model_selection`. We used a **80:20 split ratio** with stratification to maintain the class distribution of the target variable (`Reached.on.Time_Y.N`) in both sets.

```
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42, stratify=y
)
```

```
Training set shape: (8799, 17)
Testing set shape: (2200, 17)
```

5. Saving the Split Data

To preserve the processed data for future use and avoid repeating preprocessing steps, all four subsets — `x_train`, `x_test`, `y_train`, and `y_test` — were saved as separate CSV files using `to_csv()`:

```
# Save the training and testing datasets
x_train.to_csv('x_train.csv', index=False)
y_train.to_csv('y_train.csv', index=False)
x_test.to_csv('x_test.csv', index=False)
y_test.to_csv('y_test.csv', index=False)
✓ 0.2s
```

NAME: Arpita Mishra

BATCH : 03